

```
[anil@shishi CharDriver]# ls /sys/class/
backlight/ firmwares/ iwlwifi/ pinctrl/ sound/
bluetooth/ misc/ iwlwifi/ power_supply/ thermal/
black/ graphics/ iwlwifi/ ppsdev/ tty/
cpu/ hidraw/ input/ rtc/ udf/
crypto/ iwlwifi/ mtd/ scsi_device/ video/
dma/ iwlwifi/ mtd/ scsi_disk/ video_output/
dmi/ ide_port/ net/ scsi_generic/ vtconsole/
dmi/ iwlwifi/ pci_bus/ scsi_host/
[anil@shishi CharDriver]# ls /sys/class/chardev/
mynull
[anil@shishi CharDriver]# ls -l /sys/class/chardev/
total 0
lrwxrwxrwx 1 root root 0 2011-02-10 11:42 mynull -> ../../devnull/chardev/mynull
[anil@shishi CharDriver]# ls -l /sys/class/chardev/mynull/
total 0
crw-rw-r-- 1 root root 4096 2011-02-10 11:41 dev
dwxr-xr-x 2 root root 0 2011-02-10 11:42 power/
lrwxrwxrwx 1 root root 0 2011-02-10 11:41 subsystem -> ../../../../class/chardev/
crw-rw-r-- 1 root root 4096 2011-02-10 11:41 vcsdev
[anil@shishi CharDriver]# cat /sys/class/chardev/mynull/dev
256 0
[anil@shishi CharDriver]# ls -l /dev/mynull
crw-rw--- 1 root root 256, 0 2011-02-10 11:41 /dev/mynull
[anil@shishi CharDriver]#
```

Figure 1: Automatic device file creation

Here, the first is `dev_t` with the corresponding `<major, minor>`. The corresponding complementary or the inverse calls, which should be called in chronologically reverse order, are as follows:

```
device_destroy(cl, first);
class_destroy(cl);
```

Refer to Figure 1 for the `/sys` entries created using 'chardev' as the `<device class name>` and 'mynull' as the `<device name format>`. That also shows the device file, created by `udev`, based on the `<major>:<minor>` entry in the `dev` file.

In case of multiple minors, the `device_create()` and `device_destroy()` APIs may be put in the `for` loop, and the `<device name format>` string could be useful. For example, the `device_create()` call in a `for` loop indexed by 'i' could be as follows:

```
device_create(cl, NULL, MKDEV(MAJOR(first), MINOR(first) + i),
NULL, "mynull%d", i);
```

File operations

Whatever system calls (or, more commonly, file operations) we talk of on a regular file, are applicable to device files as well. That's what we say: a file is a file, and in Linux, almost everything is a file from the user-space perspective. The difference lies in the kernel space, where the virtual file system (VFS) decodes the file type and transfers the file operations to the appropriate channel, like a filesystem module in case of a regular file or directory, and the corresponding device driver in case of a device file. Our discussion focuses on the second case.

Now, for VFS to pass the device file operations onto the driver, it should have been informed about it. And yes, that is what is called registering the file operations by the driver with the VFS. This involves two steps. (The parenthesised

```
[anil@shishi CharDriver]# dmesg
hamamaker: ofcd registered
[anil@shishi CharDriver]# ls -l /dev/mynull
crw-rw--- 1 root root 256, 0 2011-02-10 14:54 /dev/mynull
[anil@shishi CharDriver]# ls
root:
[anil@shishi CharDriver]# echo "Hello Universe" > /dev/mynull
[anil@shishi CharDriver]# dmesg | tail -10
hamamaker: ofcd registered
Driver: open()
Driver: write()
Driver: close()
[anil@shishi CharDriver]# cat /dev/mynull
[anil@shishi CharDriver]# dmesg | tail -10
hamamaker: ofcd registered
Driver: open()
Driver: write()
Driver: close()
Driver: open()
Driver: read()
Driver: close()
[anil@shishi CharDriver]#
```

Figure 2: "null driver" experiments

code refers to the 'null driver' code below.) First, let's fill in a file operations structure (`struct file_operations pugs_fops`) with the desired file operations (`my_open`, `my_close`, `my_read`, `my_write`, ...) and initialise the character device structure (`struct cdev c_dev`) with that, using `cdev_init()`. Then, hand this structure to the VFS using the call `cdev_add()`. Both `cdev_init()` and `cdev_add()` are declared in `<linux/cdev.h>`. Obviously, the actual file operations (`my_open`, `my_close`, `my_read`, `my_write`) also had to be coded. So, to start with, let's keep them as simple as possible—let's say, as easy as the 'null driver'.

The null driver

Following these steps, Shweta put the pieces together, attempting her first character device driver. Let's see what the outcome was. Here's the complete code; find the file `ofcd.c` [at URL <http://bit.ly/enT4L5>]:

```
#include <linux/module.h>
#include <linux/version.h>
#include <linux/kernel.h>
#include <linux/types.h>
#include <linux/kdev_t.h>
#include <linux/fs.h>
#include <linux/device.h>
#include <linux/cdev.h>

static dev_t first; // Global variable for the first device number
static struct cdev c_dev;
// Global variable for the character device structure
static struct class "cl"; // Global variable for the device class

static int my_open(struct inode *i, struct file *f)
{
    printk(KERN_INFO "Driver: open()\n");
    return 0;
}

static int my_close(struct inode *i, struct file *f)
```